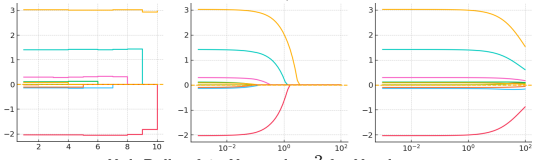
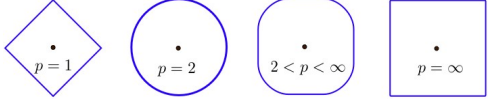


Regularization Path: Shows how model coefficients for **Best Subset**, **Lasso**, and **Ridge** change as λ increases (higher penalty $\rightarrow$ more coefficients shrink):



Unit Balls of ℓ_p Norms in $\mathbb{R}^2$ for Varying p :



Hyperparameter Tuning

- **K-Fold Cross-Validation:**
- **Unsupervised Learning**

Clustering

K-Means

- **Goal:** Partition m samples into k disjoint clusters so that points are *close as possible* to their respective cluster centroid.
- **Objective:** Minimize $\sum_j \sum_{x \in C_j} \|x - \mu_j\|^2$ (sum of squared distances to centroids).
- **Optimal clustering is NP-hard:** Cannot be solved exactly in polynomial time; K-Means is a heuristic that finds a local minimum.

K-Means Algorithm (Lloyd's)

Input: $X = \{x_1, \dots, x_m\} \subset \mathbb{R}^d, k \in \mathbb{N}_+$
Randomly choose initial centroids $\mu_1, \dots, \mu_k$
repeat
 (Assignment)
 $C_j \leftarrow \{x_i \mid j = \arg \min_n \|x_i - \mu_n\|_2^2\} \quad \forall j$
 (Update) $\mu_j \leftarrow \frac{1}{|C_j|} \sum_{x \in C_j} x \quad \forall j$
until centroids unchanged **or** assignments no longer move points
Return: clusters $C_1, \dots, C_k$

- **Convergence:** Guaranteed to terminate in $\mathcal{O}(m)$ iterations, each strictly reducing G , but only to a *local* minimum.
- **Sensitivity:** Result depends heavily on initial centroids;
- **Robustness:** Use *multiple random restarts* for better results.
- **Bias-variance trade-off:** Higher k reduces bias, increases variance.

Spectral Clustering

REVISIT

- **Goal:** Detect clusters based on the *connectivity structure* of the data rather than purely geometric proximity.
- **Affinity graph:** Build a weighted, undirected graph on samples $X = \{x_1, \dots, x_m\} \subset \mathbb{R}^d$. For a bandwidth hyperparameter $\varepsilon > 0$,
$$A_{ij} = \exp\left(-\frac{\|x_i - x_j\|_2^2}{\varepsilon}\right) \quad (\text{often sparsified / } k\text{-NN}).$$

$A \in \mathbb{R}^{m \times m}$ is the *affinity matrix*.
• **Graph Laplacian (unnormalised):**
$$D = \text{diag}\left(\sum_j A_{ij}\right), \quad L = D - A.$$

- (A normalised version $L_{\text{Sym}} = D^{-1/2} L D^{-1/2}$ is common as well.)
- **Key spectral fact:** number of connected components equals the multiplicity of eigenvalue 0 of L (spectral graph theory).

Spectral Clustering Algorithm (simplified)

Input: data X , bandwidth $\varepsilon > 0$, desired clusters k
Build affinity matrix $A_{ij} = \exp(-\|x_i - x_j\|_2^2 / \varepsilon)$
Compute degree matrix $D = \text{diag}(\sum_j A_{ij})$
Form unnormalised Laplacian $L = D - A$
Compute k eigenvectors $u_1, \dots, u_k$ of L with *smallest* eigenvalues
Stack $U = [u_1, \dots, u_k] \in \mathbb{R}^{m \times k}$ (i-th row $\rightarrow$ embedding of x_i)
Apply K-Means to the rows of U to obtain clusters $C_1, \dots, C_k$
Return: $C_1, \dots, C_k$

- **No random initialisation** of centroids (randomness only from graph sparsification or K-Means post-step).
- **Hyperparameters:** bandwidth ε and affinity type (heat kernel, k -NN graph, etc.).
- **Choosing k :** count eigenvalues of L (or L_{Sym}) that are *close to zero* (eigengap heuristic).
- **Properties:**
 - Captures *non-convex* or *manifold* clusters.
 - More robust to noise than naive connectivity methods.
 - ε acts as a bias-variance knob (large $\varepsilon \Rightarrow$ smoother, fewer clusters).

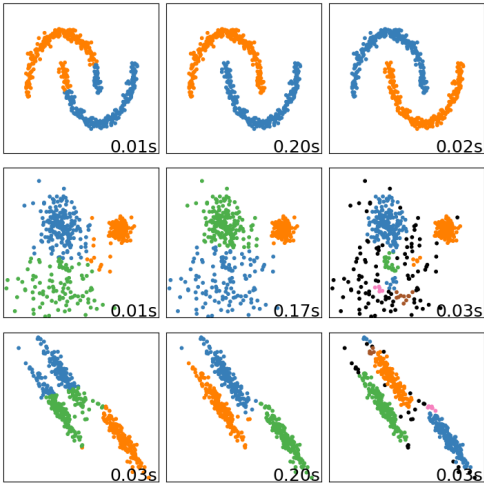
DBSCAN

Density-based clustering that groups together points that are close to each other.

Agglomerative Clustering

Hierarchical clustering that merges clusters based on distance.

KMeans vs. Spectral Clustering vs. DBSCAN



Dimensionality Reduction

PCA (Principal Component Analysis)

- **Goal:** Find k orthogonal directions (principal components) capturing maximal variance.
- **Formulation:** $W \in \mathbb{R}^{k \times d}, W^T W = I_k$
- **Optimization:** $w^* = \arg \max \{v_1, \dots, v_k\} \sum_{j=1}^k \text{Var}(v_j^T X)$
- **Covariance Matrix:** $\Sigma = \frac{1}{m} \sum_{i=1}^m (x_i - \bar{x})(x_i - \bar{x})^T$
- **Principal Components:** Top k eigenvectors of Σ (correspond to largest eigenvalues)
- **Projection:** $x \mapsto W^T (x - \bar{x})$
- **Properties:** $\|x - v\|_2 \geq \|x - Px\|_2$ for $v \notin \text{span}(u_1, \dots, u_k)$
- **Explained Variance:** λ_j is variance explained by u_j ; $\sum_j \lambda_j$ is total variance.

PCA Algorithm (Covariance-based)

Input: $X \in \mathbb{R}^{m \times d}$ (design matrix with m samples, d features)
Compute empirical mean: $\bar{x} \leftarrow \frac{1}{m} \sum_{i=1}^m x_i$
Center the data: $Z \leftarrow X - 1\bar{x}^T$
Compute covariance matrix: $\Sigma \leftarrow \frac{1}{m} Z^T Z$
Eigen-decompose Σ ; keep top k eigenvectors $u_1, \dots, u_k$
Return: $u_1, \dots, u_k$ (principal component directions)

Vectors $u_1, \dots, u_k$ are orthonormal and maximise retained variance.

Gradient Based Learning

Basic Idea use an iterative optimization algorithm to minimize a loss function by updating parameters in the opposite direction of the gradient.

Sub-gradient Concepts

- **Sub-gradient:** v is a sub-gradient of f at w if $f(u) \geq f(w) + \langle v, u - w \rangle$ for all u .
- **Sub-differential:** $\partial f(w)$ is the set of all sub-gradients of f at w .
- **Convexity:** f is convex if $f(tv + (1-t)u) \leq tf(v) + (1-t)f(u)$ for $t \in [0, 1]$.
- **Examples:** Halfspace, norm, sum of convex functions.
- **Directional derivative:** $f(y) \geq f(x) + \langle \nabla f(x), y - x \rangle$ for differentiable f .

Backtracking Line Search Algorithm

Initialize: $\eta = 1$
while $f(x) + \alpha \eta \langle \nabla f(x), \Delta x \rangle < f(x + \eta \Delta x)$ **do**
 $\eta \leftarrow \beta \eta \quad (\beta \in (0, 1))$
end while
Return η

Sub-gradient Descent Algorithm

Initialize: $w^{(1)} = 0$
for $t = 1, \dots, T$ **do**
 Choose $v_t \in \partial f(w^{(t)})$
 $w^{(t+1)} \leftarrow w^{(t)} - \eta_t v_t$
end for
Return $\bar{w} = \frac{1}{T} \sum_{t=1}^T w^{(t)}$

Stochastic Gradient Descent (SGD) Algorithm

Initialize: $w^{(1)} = 0$
for $t = 1, \dots, T$ **do**
 Choose $(x, y) \sim \mathcal{D}$
 Choose $v_t \in \partial \ell(w^{(t)}, (x, y))$
 $w^{(t+1)} \leftarrow w^{(t)} - \eta_t v_t$
end for
Return $\bar{w} = \frac{1}{T} \sum_{t=1}^T w^{(t)}$

Neural Networks

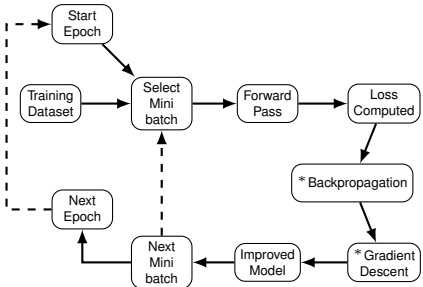
Backpropagation

"Backpropagation is the algorithm for determining how a single training example would like to nudge the weights and biases (Gradient-based update), not just in terms of whether they should increase or decrease, but also in terms of the relative proportions that would cause the most rapid decrease in the cost function (Gradient direction). A true gradient descent step would involve computing these adjustments for all tens of thousands of training examples and averaging the resulting gradients. However, this is computationally expensive, so instead, the training data is randomly subdivided into mini-batches, and each gradient descent step is computed with respect to one mini-batch at a time. By repeatedly going through all of the mini-batches (Epoch) and updating the weights accordingly, the network gradually converges toward a local minimum of the cost function. In doing so, the network ends up performing very well on the training examples."

Useful Terms:

- **Mini-batch:** A small subset of the training data used to compute the gradient for a single update step.
- **Batch Size:** The number of training examples in a mini-batch (hyperparameter).
- **Epoch:** One complete pass through the entire training dataset (via all mini-batches).
- **Dropout (rate):** A regularization technique where a fraction of neurons are randomly set to zero during a batch training to prevent overfitting. The rate is an hyperparameter (usually 50%).

Training Pipeline:



* Backpropagation computes gradients; a gradient-based optimiser applies them over many mini-batches and epochs.

FNNs (Feedforward Neural Networks)

CNNs (Convolutional Neural Networks)

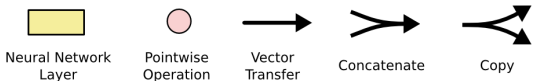
RNNs (Recurrent Neural Networks)

RNNs process sequential data, using previous steps to inform current predictions. Useful for text, audio, etc. and supports one/many-to-one/many sequence I/O patterns (e.g image $\rightarrow$ description is one-to-many).

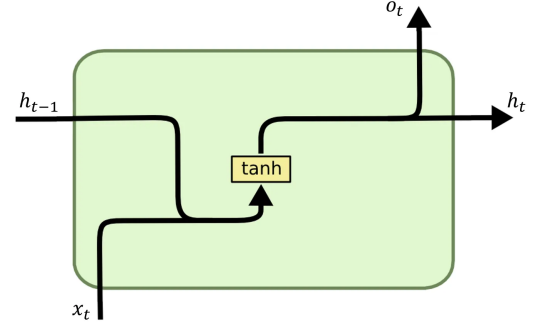
$h_t = \tanh(Uh_{t-1} + Vx_t)$

Basic RNN

Diagrams notation:

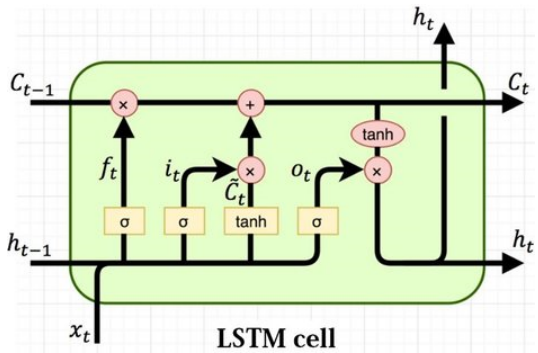


RNN Cell:



Long Short-Term Memory (LSTM)

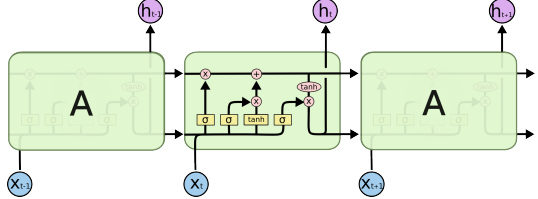
LSTMs are a type of RNN designed to remember information for long periods, addressing the vanishing gradient problem. They achieve this through a cell state (C_t) and three gates: input (i_t), forget (f_t), and output (o_t).



Where:

- $i_t = \sigma(x_t U^i + h_{t-1} W^i)$ Input gate, controls how much of the input to let through.
- $f_t = \sigma(x_t U^f + h_{t-1} W^f)$ Forget gate, controls how much of the previous cell state to keep.
- $o_t = \sigma(x_t U^o + h_{t-1} W^o)$ Output gate, controls how much of the cell state to output.
- $\tilde{c}_t = \tanh(x_t U^g + h_{t-1} W^g)$ Candidate cell state, potential new information to add to the cell state.
- $c_t = \sigma(f_t * C_{t-1} + i_t * \tilde{C}_t)$ Cell state, memory of the LSTM.
- $h_t = \tanh(C_t) * o_t$ Hidden state, output of the LSTM at time t .

LSTM Chain:



RNNs vs. CNNs

Property	RNN	CNN
Long-range mechanism	Gating/Attention	Pooling
Sequential	+	-
Parallelizable	-	+
Compute time for n tokens	$\mathcal{O}(n \cdot L)$	$\mathcal{O}(L)$
Layers required for the n -th token	1	$\log n$